Artigo

Invista em você! Saiba como a DevMedia pode ajudar sua carreira.

Tipos de dados por valor e por referência em Java

Veja neste artigo a definição de variáveis e os tipos de dados por valor e por referência utilizados na linguagem Java, entendendo como funciona sua declaração e inicialização.

Anotar

Marcar como concluído

No desenvolvimento de programas sempre precisamos guardar na memória valores, seja um nome, uma data, ou qualquer outro tipo de informação. Sendo assim, são declaradas o que conhecemos por variáveis que atribuem estes valores em tempo de execução do sistema.

Variáveis

As variáveis são posições na memória do computador que podem armazenar dados. As variáveis são formadas por quatro elementos: **nome, tipo, tamanho e valor**.

Dependendo da programação, o básico de uma declaração de variável pode ter somente um tipo, um nome e um valor.

Listagem 1: Exemplo de declaração de variável

```
public class Imprime_Variaveis {

        public static void main(String[] args) {

                int valorA = 33;
                int valorB = 25;

                int total = valorA + valorB;

                System.out.printf("A soma total = %d",total);
        }
}
```

O Java possui dois tipos de dados que são divididos em **por valor** (tipos primitivos) e **por referência** (tipos por referência).

Os tipos primitivos são **boolean, byte, char, short, int, long, float e double**. Os tipos por referência, são classes que especificam os tipos de objeto **Strings, Arrays Primitivos e Objetos**.

Uma variável do tipo primitivo pode armazenar exatamente um valor de seu tipo declarado por vez, quando outro valor for atribuído a essa variável, seu valor inicial será substituído.

As variáveis de instância de tipo primitivo são inicializadas por padrão, as variáveis dos tipos byte, char, short, int, long, float e double são inicializadas como 0, e as variáveis do tipo boolean são inicializadas como **false**. Esses tipos podem especificar seu próprio valor inicial para uma variável do tipo primitivo atribuindo à variável um valor na sua declaração.

O Java fornece dois tipos primitivos para armazenar números de ponto flutuante na memória, o tipo float e double.

A diferença entre eles é que as variáveis double podem armazenar números com maior magnitude e mais detalhes, ou seja, armazena mais dígitos à direita do ponto de fração decimal, do que as variáveis float. As variáveis do tipo float representam números de ponto flutuante de precisão simples e podem representar até 7 dígitos.

As variáveis do tipo double representam números de ponto flutuante de precisão dupla, onde precisam duas vezes a quantidade de memória das variáveis float fornecendo 15 dígitos, sendo o dobro da precisão de variáveis float. Os valores do tipo double são conhecidos como literais de ponto flutuante. Para números de ponto flutuante precisos, o Java fornece a classe BigDecimal (pacote java.math).

Listagem 2: Exemplo dos tamanhos de tipos primitivos

```
        public static void main(String[] args) {
                System.out.println("Tipos de dados em Java: \n" +
                    "\nMenor Byte: " + Byte.MIN_VALUE +
                    "\nMaior Byte: " + Byte.MAX_VALUE +
                    "\nMenor Short Int: " + Short.MIN_VALUE +
                    "\nMaior Short Int: " + Short.MAX_VALUE +
                    "\nMenor Int: " + Integer.MIN_VALUE +
                    "\nMaior Int: " + Integer.MAX_VALUE +
                    "\nMenor Long: " + Long.MIN_VALUE +
                    "\nMaior Long:" + Long.MAX_VALUE +
                    "\nMenor Float: " + Float.MIN_VALUE +
                    "\nMaior Float: " + Float.MAX_VALUE +
                    "\nMenor Double: " + Double.MIN_VALUE +
                    "\nMaior Double: " + Double.MAX_VALUE);

        }

}
```

Listagem 3: Declaração dos tipos primitivos

```
public class Tipos_Primitivos {
        public static void main(String[] args) {
              byte tipoByte = 127;
              short tipoShort = 32767;
              char tipoChar = 'C';
              float tipoFloat = 2.6f;
              double tipoDouble = 3.59;
```

```
            long tipoLong = 9223372036854775807L;
            boolean tipoBooleano = true;
            System.out.println("Valor do tipoByte = " + tipoByte);
            System.out.println("Valor do tipoShort = " + tipoShort);
            System.out.println("Valor do tipoChar = " + tipoChar);
            System.out.println("Valor do tipoFloat = " + tipoFloat);
            System.out.println("Valor do tipoDouble = " + tipoDouble);
            System.out.println("Valor do tipoInt = " + tipoInt);
            System.out.println("Valor do tipoLong = " + tipoLong);
            System.out.println("Valor do tipoBooleano = " + tipoBooleano);
        }
}
```

Na Listagem 3 o tipo declarado como char é sempre declarado com aspas simples porque o tamanho é somente de 1 caractere. Os tipos float sempre irão possuir o caractere “f” no final do valor para sua identificação, sendo a mesma coisa com o tipo long só que é inserido o caractere “L”.

Tipos por referência

Os programas utilizam as variáveis de tipos por referência para armazenar as localizações de objetos na memória do computador. Esses objetos que são referenciados podem conter várias variáveis de instância e métodos dentro do objeto apontado.

Para trazer em um objeto os seus métodos de instância, é preciso ter referência a algum objeto. As variáveis de referência são inicializadas com o valor “**null**” (nulo).

Por exemplo, ClasseConta acao = new ClasseConta(), cria um objeto de classe ClasseConta e a variável acao contém uma referência a esse objeto ClasseConta, onde poderá invocar todos os seus métodos e atributos da classe. A palavra chave new solicita a memória do

sistema para armazenar um objeto e inicializa o objeto.

```
public class AcessaMetodo {

        public void imprime(){
                System.out.println("Bem Vindo ao Java!");
        }

        public static void main(String[] args) {
                AcessaMetodo acessa = new AcessaMetodo ();
                acessa.imprime();

        }

}
```

A saída desse código acima irá ser reproduzida através da ação acessa.imprime(), porque está sendo acessado o método do objeto que foi inicializado com a variável definida como "acessa".

Considerações finais

- As variáveis de tipos por valor não referenciam objetos, esses tipos de variáveis não podem ser utilizadas para invocar métodos.
- Lembrar de que as variáveis locais não são inicializadas por padrão (variáveis dentro dos métodos).
- As variáveis do tipo primitivo não podem ser inicializadas como referência a um objeto.

Espero que tenham gostado, e até a próxima!